The Active Record way claims that intelligence belongs in your models, not in the database. As such, features such as triggers or constraints, which push some of that intelligence back into the database, are not heavily used.

Validations such as `validates :foreign_key, uniqueness: true` are one way in which models can enforce data integrity. The `:dependent` option on associations allows models to automatically destroy child objects when the parent is destroyed. Like anything which operates at the application level, these cannot guarantee referential integrity and so some people augment them with foreign key constraints in the database.
Although Active Record does not provide all the tools for working directly with such features, the `execute` method can be used to execute arbitrary SQL.

8 Migrations and Seed Data

Some people use migrations to add data to the database:

```
class AddInitialProducts < ActiveRecord::Migration
  def up
    5.times do |i|
      Product.create(name: "Product ##{i}", description: "A product.")
    end
  end

  def down
    Product.delete_all
  end
end
```

However, Rails has a 'seeds' feature that should be used for seeding a database with initial data. It's a really simple feature: just fill up `db/seeds.rb` with some Ruby code, and run `rake db:seed`:

```
5.times do |i|
  Product.create(name: "Product ##{i}", description: "A product.")
end
```

This is generally a much cleaner way to set up the database of a blank application.